

Edwin Duran
ECE 157C
Due: 05/01/26

HW2 REPORT

Execution Result Design

The system represents execution results as JSON-compatible structured objects. The generated analysis code always assigns its computed output to a variable named `result`. After execution, `execute_code_node` stores that value in the agent graph state as `execution_result`. The result can be a scalar value, a dictionary, a list, or a table-like list of dictionaries. This makes the output flexible enough to support simple numeric answers, grouped summaries, ranked rows, and more complex explanatory outputs.

The result is then persisted by `backend/result_store.py`. That file converts common Python and pandas objects into JSON-safe forms before saving them. For example, a pandas DataFrame is converted into a list of row dictionaries, a Series is converted into a dictionary, and numpy scalar values are converted into plain Python scalar values when possible. Each completed turn is saved as a full JSON record in `stored_results/`. The system also writes a row to `results.csv` so there is a compact index of previous runs.

The saved JSON record contains more than just the final answer. It stores the original question, resolved question, whether prior context was used, generated code, `execution_result`, `final_answer`, evaluation status, visualization decision, visualization result, visualization HTML, and errors if any occurred. This makes the stored record useful as a clean interface between components and also useful as a debugging artifact.

This representation was chosen because JSON is portable and inspectable. It can be sent through backend responses, saved to disk, loaded again later, and shown in debugging tools without requiring the original dataset. The frontend and backend both already communicate using JSON, so using JSON-compatible execution results reduces translation complexity. The CSV index is useful for quickly seeing what questions were asked and where the full stored result lives, while the JSON files preserve the complete structured result.

The representation also supports follow-up queries. When a user asks for a follow-up such as "Now show top 5" or "Filter the previous output," the system passes the prior `execution_result` into the next turn as `stored_result`. The follow-up code generation prompt instructs the model to use `stored_result`, `prior_turn`, and `conversation_history` as the only data sources. The executor also rejects follow-up code that tries to access the dataset with `pd.read_csv`, `csv_path`, `open`, or `Path`. This means follow-up operations are intended to operate on the previous stored result rather than recomputing from the raw CSV.

The main trade-off is that JSON does not preserve every Python or pandas type exactly. DataFrames lose their DataFrame methods once converted into records, and some unusual objects may be stringified. However, this is acceptable because the purpose of the representation is to create a clean boundary between components, not to preserve every internal Python type. Another trade-off is storage size. If a result contains many rows or a visualization contains large HTML, the stored JSON files can become large. The benefit is that the record becomes easier to inspect, reproduce, and reuse. A final trade-off is that follow-up quality depends on the structure of the previous `execution_result`. If the first answer only stores a short string, then later operations like sorting or filtering may not have enough structured information to work with. For this reason, the code generation prompt encourages JSON-serializable structured outputs.

Visualization Agent Interface

The visualization component is implemented in `nodes.py` using three main nodes: `assess_visualization_node`, `generate_visualization_code_node`, and `execute_visualization_code_node`. These nodes form a separate visualization stage after the system has generated and verified an answer.

The input state of the visualization component includes the user question, the computed `execution_result`, the final natural-language answer, `prior_turn` context, and `visualization_decision`. The most important design decision is that the visualization component treats `execution_result` as its data interface. It does not read the original CSV dataset. This keeps visualization separate from analysis: the analysis component computes a result, and the visualization component decides how to display that result.

`assess_visualization_node` decides whether visualization is useful. It considers the question, the computed result, and the final answer. It returns a structured decision with four fields: `needed`, `chart_type`, `reason`, and `data_focus`. Visualization is optional. If the result is a single scalar, the component usually decides that a chart is not necessary

unless the user explicitly asks for one. If the result compares categories, ranks values, shows distributions, or describes relationships, the component can request a chart.

`generate_visualization_code_node` generates Plotly code only when visualization is needed. The generated code must use Plotly Express or Plotly Graph Objects. The prompt instructs the code to use `result`, `stored_result`, `prior_turn`, and `visualization_decision` as the only data sources. It also explicitly forbids reading the original CSV, opening local files, using `Path`, or using `csv_path`. This keeps the design aligned with the requirement that visualization operates on the execution result rather than the raw dataset.

`execute_visualization_code_node` executes the generated plotting code. It provides `result`, `stored_result`, `prior_turn`, and `visualization_decision` in the execution environment. It does not provide `csv_path` as a usable data source. It also has a runtime guard that rejects visualization code containing `pd.read_csv`, `read_csv`, `open`, `Path`, or `csv_path`. This guard is important because the code is generated by an LLM, and prompts alone are not enough to enforce the system boundary.

The output state of the visualization component includes `visualization_html`, `visualization_result`, `visualization_error`, `generated_visualization_code`, and `visualization_decision`. `visualization_html` is the main frontend output. It is generated with Plotly's `fig.to_html` and embedded into the frontend chat response inside an `iframe`. If no visualization is generated, the frontend simply displays the final answer. If visualization generation fails, the error is stored in the result record and the answer can still be displayed without a chart.

System Usage and Summary

The system was designed for a chatbot-style workflow. A user selects a dataset, asks a question, receives a final answer, and optionally receives a visualization. The user can then ask follow-up questions that refer to the previous result. The frontend maintains chat sessions, and the backend maintains per-session conversation history while the server is running. The frontend also stores session display history in `localStorage` so browser reloads restore previous chats and open a new active session.

Example questions tested with the system:

1102
Identify students with similar study time but significantly different academic performance. What factors might explain these differences?

AGENT

There are 137 student pairs with very similar study/AI usage time, <= 0.3 hours difference, but a large difference in academic impact, at least 2 performance levels apart.

Key takeaway:

- Similar study time alone does not explain performance differences.
- The most likely explaining factors are:
 - Age: 123 pairs
 - AI tool used: 115 pairs
 - City: 115 pairs
 - Purpose of use: 111 pairs
 - Education level: 91 pairs
 - Satisfaction level: 86 pairs
 - Gender: 72 pairs

Examples:

- Student 1 vs 57
 - Both used about 0.8 hours/day
 - Student 1: Slight Decline
 - Student 57: Improved
 - Likely differences: age, city, AI tool used
- Student 4 vs 8
 - Both used 4.2 hours/day
 - Student 4: Improved
 - Student 8: Slight Decline
 - Likely differences: age, AI tool used, purpose
- Student 3 vs 7
 - 1.8 vs 2.0 hours/day
 - Student 3: Improved
 - Student 7: Slight Decline
 - Likely differences: age, gender, education level, city, AI tool used

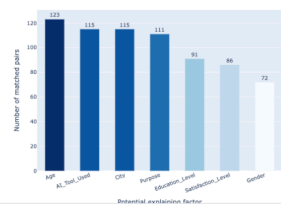
What might explain the differences?

- How the tool is used matters more than just time spent.
- Students using AI for learning, homework, or research may get different outcomes than those using it mainly for coding or writing.
- Different AI tools may support different tasks better.
- Education level and age may affect how effectively students use the tools.
- Satisfaction level may reflect whether the tool is actually helping the student.
- City and gender may also capture differences in access, study environment, or learning habits.

Overall, the strongest pattern is that performance differences among students with similar study time are most often associated with age, AI tool choice, city, and purpose of use.

Most Common Explaining Factors

Across 137 matched student pairs with similar study time but different performance



1103
now show only top 5 potential explaining factors

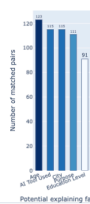
AGENT

Top 5 most common potential explaining factors:

- Age — 123
- AI Tool Used — 115
- City — 115
- Purpose — 111
- Education Level — 91

Top 5 Explaining Factors

Among 137 matched student pairs with similar study time



Info

Find students who have high stress levels despite relatively low study hours. What factors might explain this?

AGENT

Using the proxy of high stress as low satisfaction and/or slight grade decline, there are 11 students who appear to have relatively high stress despite low daily usage of AI tools (<= 1.8 hours).

Who they are:

- Student IDs: 1, 3, 24, 30, 38, 41, 43, 57, 66, 79, 88

What stands out:

- Low satisfaction is the strongest signal
- 8 of 11 had Low satisfaction
- Academic outcomes were not strongly positive:
- 4 had Slight Decline in grades
- 4 had No Change
- 3 had Improved grades
- Most common usage purposes:
- Writing: 4
- Research: 2
- Learning: 2
- Coding: 2
- Homework: 1

Possible factors that might explain the stress:

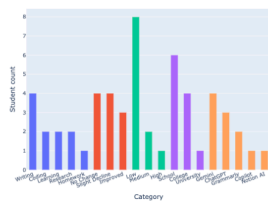
- Dissatisfaction with AI support seems important, since most of these students reported Low satisfaction.
- Limited academic support may be contributing.
- Most students saw either no improvement or a slight decline in grades.
- Writing-related use appears most common, which may suggest students are using AI for assignment pressure rather than deeper learning support.
- The group is concentrated more in:
 - School level: 6 students
 - College level: 4 students
- Tool use is spread across several platforms, but Gemini and ChatGPT appear most often:
 - Gemini: 4
 - ChatGPT: 3

Important caveat:

- The dataset does not include a direct stress variable, so this is an inferred group rather than confirmed high-stress students.

Potential factors among low-study-hour, high-stress proxy

Identified subgroup size: 11 students | Low-usage threshold: <= 1.8 hours



Info

Identify the top 5 student groups with the highest academic performance and explain the key factors contributing to their success.

AGENT

The top 5 student groups with the highest academic performance all achieved the same performance score of 4.9.

- University students using Gemini for Research

- 2 students

- Avg grade impact: 2.0

- Improved grade rate: 100%

- Avg satisfaction: 3.0

- Avg daily usage: 3.4 hours

- School students using Nelson AI for Coding

- 1 student

- Avg grade impact: 2.0

- Improved grade rate: 100%

- Avg satisfaction: 3.0

- Avg daily usage: 5.0 hours

- School students using ChatGPT for Learning

- 1 student

- Avg grade impact: 2.0

- Improved grade rate: 100%

- Avg satisfaction: 3.0

- Avg daily usage: 5.0 hours

- School students using Gemini for Learning

- 1 student

- Avg grade impact: 2.0

- Improved grade rate: 100%

- Avg satisfaction: 3.0

- Avg daily usage: 2.7 hours

- University students using Copilot for Homework

- 1 student

- Avg grade impact: 2.0

- Improved grade rate: 100%

- Avg satisfaction: 3.0

- Avg daily usage: 3.7 hours

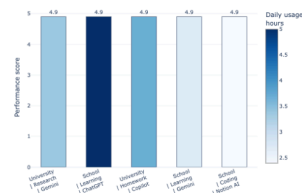
Key factors contributing to their success:

- Positive academic impact: every top group reported a strong positive grade impact.
- 100% improvement rate: all students in these groups said their grades improved.
- High satisfaction: each group showed strong satisfaction with the AI tool they used.
- Effective usage habits: success came either from steady engagement, around 3 to 5 hours daily, or efficient use without needing excessive time.
- Clear task-tool fit: high-performing groups used AI tools for specific purposes such as research, coding, learning, and homework, suggesting focused use helps results.

One important note:

- Most of these groups have very small sample sizes, with 1 student each except the University-Research-Gemini group with 2 students, so the results are promising but should be interpreted cautiously.

Top 5 Student Groups by Academic Performance





Summary of insights:

These results suggest that academic outcomes are not explained by study time or AI usage time alone. Students with nearly identical study or AI-use hours can have very different grade outcomes, which means the quality and context of AI use matter more than raw time spent. The most common differentiating factors were age, AI tool used, city, purpose of use, education level, satisfaction level, and gender. This suggests performance differences may come from how students use AI, what tools they use, what academic tasks they use it for, and differences in learning environment or background.

The stronger-performing groups tended to report improved grades and often used AI for focused academic purposes such as research, coding, learning, or homework. In contrast, students with high-stress indicators despite low study hours often showed low

satisfaction or limited grade improvement, suggesting that simply using AI does not guarantee better outcomes. Overall, the system's results point to a key theme: AI is most helpful when it is used purposefully and effectively, not merely used for more hours.

Failure Case

One failure case observed during development involved multiple chat sessions in the frontend. The user could open two sessions, ask a question in Session 1, immediately switch to Session 2 while the agent was still thinking, and then wait. When the backend finished, the answer intended for Session 1 appeared in Session 2. This was incorrect because the response belonged to the session that submitted the request, not the session that happened to be active later.

What went wrong was that the frontend rendering logic originally depended on the current active session at response time. The backend request contained the correct `session_id`, but the frontend had a single global pending message and used `currentSession()` when adding the final agent response. Since `currentSession()` changes when the user clicks a different session, the final response could be appended to the wrong local chat history.

This happened because asynchronous requests finish later than they are started. At send time, the user was in Session 1. At response time, the user might be in Session 2. The old code did not preserve the originating session id throughout the entire request lifecycle. This made the UI state race against the network/agent execution time.

The system was improved by capturing `requestSessionId` and `requestDataset` at the exact moment the user sends the question. The pending message, final answer, visualization, and error message are now all routed using `requestSessionId`. The frontend functions `addSessionMessage`, `renderUserMessage`, `renderAgentMessage`, `renderPendingAgentMessage`, and `removePendingAgentMessage` now accept a session id instead of assuming the active session. Therefore, if the user submits a question in Session 1 and switches to Session 2, the response is still stored in Session 1 and appears there when the user returns.

This fix improves frontend memory consistency, but there are still possible limitations. The frontend stores display history in `localStorage`, while the backend stores active session history in memory. If the backend server restarts, the browser may still show old chat messages, but the backend may not have the full corresponding session memory unless it is rebuilt from stored results or persisted separately. A future improvement

would be to store backend session histories in a database or JSON file and reconnect restored frontend sessions to backend memory after reloads or server restarts.